Sun-damage prediction on the 3D avatar

Optional visualisation add-on for the `photodamage-dermlip-mtl-v4` model. It takes the model's **per-tile predictions** and paints them onto the patient's **Vectra 3D body avatar**, producing:

Output	What it is
`raw_avatar.glb` / `.ply`	the photo-textured 3D body (no model output)
`raw_turntable.gif`	the raw avatar rotating 360°
`sundamage_avatar.glb` / `.ply`	the body coloured by predicted sun-damage severity (green → amber → red)
`sundamage_turntable.gif`	the severity avatar rotating, with a Mild → Moderate → Severe colour-bar
`sundamage_avatar_panel.png`	front/left/back/right report figure with colour-bar + caption
`sundamage_{front,left,back,right}.png`	the four still views
`vertex_severity.npy`	per-vertex severity (0–2), for custom figures

Severity is the probability-weighted class value `severity = 1·P(moderate) + 2·P(severe) ∈ [0, 2]` — the same `severity_score` the model uses per patient, here resolved per body location.

What you need (prerequisites)

This is **not** self-contained like the tile inference — it needs the raw Vectra capture for the patient, because the 3D geometry and camera calibration live there. You must supply **two inputs**:

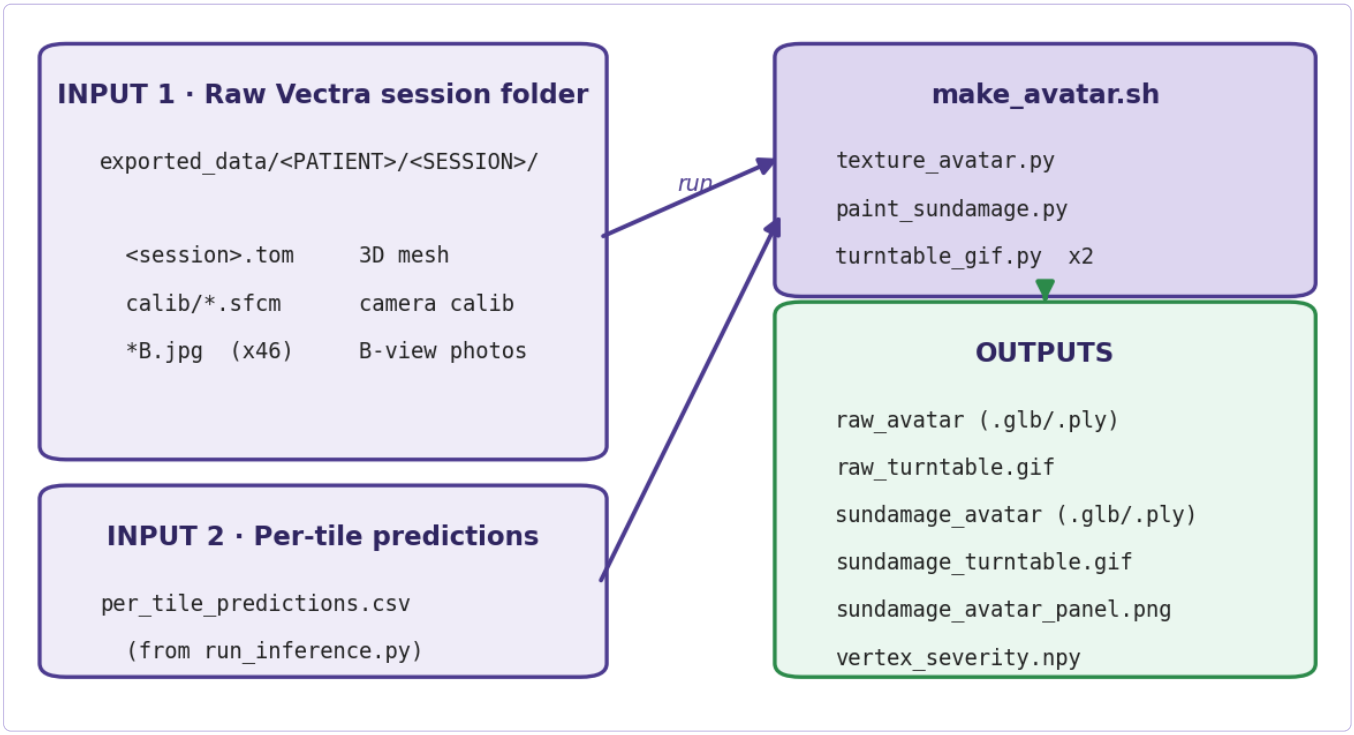
Input	What / where it comes from
Raw Vectra session folder	the patient's baseline capture dir, holding `<session>.tom` (3D mesh), `calib/*.sfcm` (per-camera calibration) and `*B.jpg` (46 B-view photos). In the study these are at `exported_data/<PATIENT>/<SESSION>/`. **Not shipped in this model package.**
Per-tile predictions CSV	`per_tile_predictions.csv` written by `run_inference.py` (the main package). One CSV may hold many patients — the driver slices one out by `patient_id`.

> If a session has no `*B.jpg` preview photos, point the raw-texture step (step 1) at a folder of the converted colour images for that patient instead.

Install the extra deps (in addition to the package's `requirements.txt`):

```
pip install -r avatar_3d_visualization/requirements.txt
```

How the two inputs feed the pipeline:



Quick start (one command)

```
cd avatar_3d_visualization
bash make_avatar.sh <session_dir> <per_tile_predictions.csv> <out_dir> [patient_id]
```

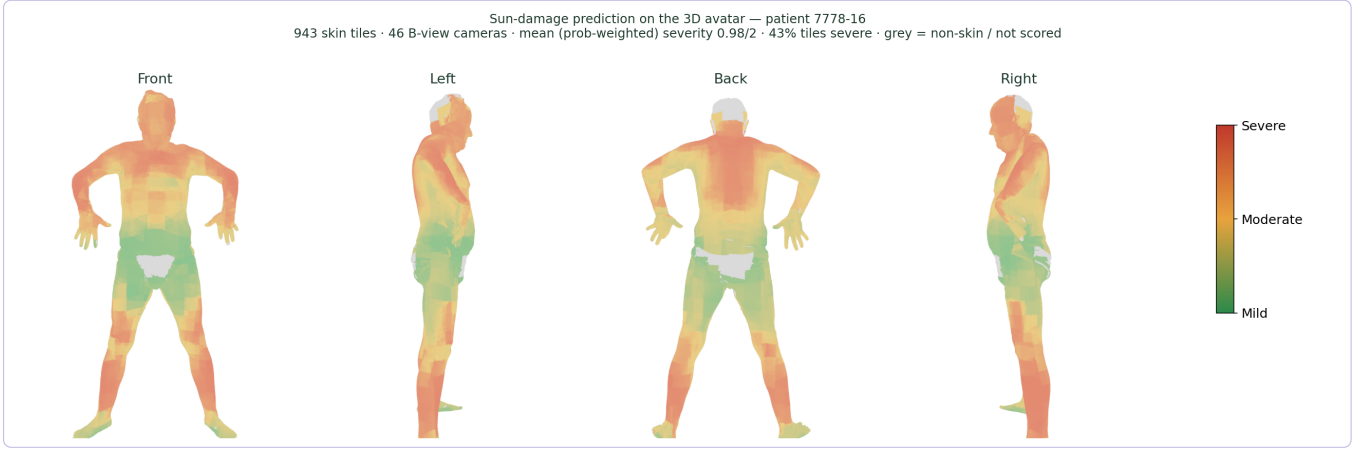
Example:

```
bash make_avatar.sh \
    "/data/exported_data/06368481-.../20221207140356" \
    ../out/per_tile_predictions.csv \
    ../out/avatar_7778-16  7778-16
```

This runs all four steps and writes every output above into `out_dir`.

Example outputs (patient 7778-16)

The **report panel** — front / left / back / right, coloured by predicted severity (green = mild → red = severe; grey = non-skin / not scored):



The two **turntable GIFs** (shown here as sampled frames — in the files they rotate a full 360°): the raw photo avatar and the severity avatar with its colour-bar.

Step by step (what the driver runs)

```
 # 1. Raw photo avatar  (image_dir = session so it uses the *B.jpg photos)
python3 texture_avatar.py   <session_dir> <image_dir> <out_dir>

# 2. Severity avatar + still views + report panel
python3 paint_sundamage.py  <session_dir> <preds_csv> <out_dir>

# 3. Raw turntable gif        (`nobar` = no severity legend)
python3 turntable_gif.py     <out_dir>/raw_avatar.ply        <out_dir>/
raw_turntable.gif        48 640 960 nobar

# 4. Severity turntable gif   (severity colour-bar baked in)
```

```
python3 turntable_gif.py     <out_dir>/sundamage_avatar.ply  <out_dir>/
sundamage_turntable.gif  48 640 960
```

`turntable_gif.py` args after the output path are: `n_frames width height [nobar]`.

How it works

`paint_sundamage.py` reuses the study's validated camera model: each mesh vertex is projected into every **B-view** camera via that camera's `.sfcm` calibration, occlusion-tested with an Open3D ray cast (so hidden surfaces aren't painted), and the visible, front-facing cameras vote. Instead of sampling the photo colour (that's `texture_avatar.py`), it looks up the **predicted severity of the tile** covering that pixel and blends by how squarely each camera faces the vertex.

The tile lookup mirrors the cropping pipeline: the camera image is rotated 90°/270° per camera and cut into a 5-column × 9-row grid (`a1` … `e9`), so the prediction CSV's `camera` + `grid_cell` map straight onto the projected pixel.

Predictions CSV — the packaged `paint_sundamage.py` accepts either: - this package's `per_tile_predictions.csv` (`pred_photodamage` string), or - the study's `04_infer_sundamage.py` output (numeric `pred`). It only needs the columns `patient_id, camera, grid_cell, p_mild, p_moderate, p_severe`.

Notes & caveats

- **Grey = not scored** — vertices no camera could see, or that fell on a tile rejected as non-skin, stay neutral grey.
- These meshes use **per-vertex colour** (unlit), so they render correctly in any viewer, including Open3D. (The separate photo-*textured* atlas GLBs render dark in Open3D — not used here.)
- The projection ignores lens distortion (`K1,K2`); it's negligible for these long-focal skin cameras and was validated visually against the photo texture.
- Runtime is ~1–2 min per patient (mostly the two turntable renders). Lower `n_frames` (e.g. 24) for smaller/faster gifs.
- Files carried here: `sfcm.py`, `tom_texture.py` (mesh loader), `texture_avatar.py`, `paint_sundamage.py`, `turntable_gif.py`, `make_avatar.sh`.